HIERARCHICAL TENANT CONFIGURATION WITH LIVE PROPAGATION

MoveInSync Backend Case Study 2
Final Design, Implementation & Verification Report

Java 21 · Spring Boot 4.1.2 · MySQL 8 · Server-Sent Events · Java client library

GitHub repository
https://github.com/ankitapaul74/tenant_config

Report scope
System design, implemented features, API verification, performance simulations, failure behavior, security, trade-offs, and requirement traceability.

Evidence statement: This report separates implemented and exercised functionality from requirements that remain unverified. It does not claim that unrun tests or missing benchmarks have passed.

1. Problem statement and solution

MoveInSync operates a multi-tenant platform where configuration values may be defined at GLOBAL, CLIENT, OFFICE, and SHIFT levels. The most specific applicable value must win. Since many services read settings frequently, they should use a local snapshot rather than making a network request for every read. Updates must propagate promptly, invalid values must be rejected, and atomic changes must not be partially visible.

The solution is a Spring Boot configuration service backed by MySQL, with a Java client library that maintains an immutable in-memory snapshot. Server-Sent Events (SSE) provide push notifications; periodic version checks provide a recovery path for missed notifications. JSON snapshot persistence supports recovery when the database is unavailable.

2. Architecture and core responsibilities

Layer / component	Responsibility
REST controllers	Configuration definitions, overrides, resolve/explain, history, version, change sets, staged rollout, promotion, rollback, SSE events.
Service layer	Hierarchy resolution, type validation, transactional change sets, audit history, rollout and rollback behavior.
Repositories and MySQL	Persistence of definitions, overrides, history and rollout records.
SSE publisher	Pushes configuration-change events to connected clients.
Java client library	Initial refresh, local getters, immutable snapshot via AtomicReference, periodic refresh, SSE listener, jittered reconnect and disk snapshot support.
Snapshot persistence	Saves/restores server-side configuration state, including definitions, overrides, rollouts and last processed change identifier.

3. Hierarchical model and consistency

Precedence: SHIFT → OFFICE → CLIENT → GLOBAL/default. A more specific value applies only to the matching tenant and context. The explain endpoint returns the resolved value and the scope that supplied it.

Consistency model: Accepted updates are persisted and versioned. Connected clients receive SSE notifications and periodically check the version endpoint; the client refresh interval is configured to five seconds. A refreshed immutable snapshot is atomically published, so an individual read sees one snapshot rather than a map being updated entry by entry. This prioritizes low-latency local reads and availability at the cost of a bounded period of possible staleness. The recorded tests do not prove a strict guarantee that every client observes every update within five seconds.

Out-of-order versions: The case study asks how a client handles version 41 arriving after version 43. Version/event-ID mechanisms exist, but a dedicated stale-version test was not recorded. This specific behavior is not claimed as proven.

4. Main update and recovery flows

Update flow: Admin authenticates → submits a definition/override/change set → service validates values and scope → transaction/history is persisted → event is published → clients refresh and atomically publish their snapshot → periodic version checks can reconcile missed events.

Client recovery flow: Client loads its saved snapshot → continues local reads → attempts server refresh and SSE connection → reconnects with jitter after interruption → periodically checks versions. The client library compiled; a complete independent end-to-end client application run was not recorded.

5. Implemented functionality and API verification

Area / API workflow	Observed result
Create typed definition: POST /api/configs	Created INTEGER key booking.max.duration, default 120, minimum 1, maximum 240.
Create CLIENT override: POST /api/configs/overrides	tenant1 CLIENT override value 100 created successfully.
Resolve and explain	Resolve returned 100; explain identified sourceLevel CLIENT.
History	History returned versioned changes with author/reason and old/new values.
OFFICE and SHIFT overrides	OFFICE value 90 and NIGHT SHIFT value 80 created; resolution returned the most-specific values.
Valid change set	Valid OFFICE change set accepted; office value changed to 85.
Invalid atomic change set	Change set containing valid value 70 followed by invalid value 500 was rejected. Subsequent resolve remained 85, confirming no partial update.
Staged rollout and promotion	Rollout targeting BLR_OFFICE_1 and BLR_OFFICE_2 created; rollout ID 6 was promoted globally. tenant2 resolved to 75.
Version endpoint	GET /api/configs/version?since=0 returned currentVersion 49 and updateAvailable true at the time of test.
Rollback	OFFICE value rolled back to historical version 1; resolved value became 90. History recorded a new rollback audit version rather than decreasing version history.
Authentication	Unauthenticated request returned 401; tenant USER write returned 403; permitted tenant read succeeded.
SSE live update	SSE connection established; user observed an update after publishing a configuration change. Last-Event-ID replay was not verified.
REST tenant isolation	Authenticated tenant1 request with tenant2 query context resolved tenant1's value (115); tenant2 resolved its own value (110).

6. Performance evidence

Simulation	Clients	Average latency	p50	p99
Reconnect-storm simulation	500	1181.25 ms	1174 ms	1241 ms
Propagation simulation	200	417.80 ms	419 ms	436 ms

The figures are recorded SSE simulation results from development. The 500-client result is a simulated reconnect-storm exercise, not a claim that 500 independent production hosts were connected. The 200-client figures are the recorded propagation simulation.

Not measured: local in-memory getter latency in microseconds, reproducible throughput benchmark, machine specifications, and a formal all-connected-clients-within-five-seconds guarantee. These are explicit gaps against the performance proof requirements.

7. Failure behavior and recovery

Failure / condition	Behavior	Evidence status
Invalid configuration value	Reject before publishing.	Invalid value 500 rejected by validation.
Partial change-set failure	Transaction applies all changes or none.	Verified: earlier valid item did not persist after later item failed.
Database unavailable	Serve restored/in-memory state; recover when database returns.	MySQL-unavailable snapshot recovery exercised; separate configuration-server kill test not recorded.
SSE interruption	Reconnect with jitter; periodic version check heals missed updates.	Mechanisms implemented; Last-Event-ID replay not verified in final Postman sequence.
Duplicate / out-of-order event	Avoid stale state replacing newer state.	Dedicated duplicate/out-of-order injection test not recorded.
Cross-tenant access	Derive tenant identity from authenticated user.	REST cross-tenant isolation exercised.
Unauthenticated or unauthorized write	Reject unauthenticated requests and restrict writes.	401 and 403 responses exercised.

8. Security

Spring Security HTTP Basic authentication is configured for local development with an ADMIN user and tenant USER accounts. Unauthenticated access returned 401. Tenant USER write attempts returned 403. REST tenant isolation was exercised: a tenant1-authenticated request supplying tenant2 as a query parameter returned tenant1's own value; tenant2 received its own value. Development credentials are not reproduced in this report and must be replaced with externally managed secrets before deployment.

9. Postman evidence and collection

The newly supplied Postman export was read successfully and contains 29 saved request entries. It is a request collection, not a response-history archive; the report therefore uses the actual request inventory together with the recorded response outcomes from the test sessions. The exported collection included local development authentication material; credentials are deliberately omitted from this report. The Postman collection itself is not required to understand the test conclusions below.

10. Requirement traceability

Case-study requirement	Status
Typed definitions, validation and global defaults	Implemented and exercised; types include STRING, INTEGER, DOUBLE, BOOLEAN, DURATION, TIME_WINDOW, ENUM and LIST.
GLOBAL → CLIENT → OFFICE → SHIFT precedence	Implemented and exercised.
Explain mode and versioned audit history	Implemented and exercised.
Local snapshot and microsecond reads	Snapshot/getters implemented; independent client E2E and microsecond benchmark not verified.
SSE push plus periodic refresh; <=5 sec bound	Mechanisms implemented; strict end-to-end bound not proven.
500-client reconnect storm and jitter	Simulation recorded; complete replay/delta proof not recorded.
Atomic change sets and no mixed reads	Transactional rejection tested; continuous-reader concurrency test not completed.
Staged rollout to two offices and promotion	Implemented and exercised.
Rollback with before/after diff	Historical rollback and audit entry exercised; returned diff not verified.
Server-down snapshot recovery	MySQL unavailable recovery exercised; separate server kill/restart test not recorded.
Authentication, roles and tenant isolation	401/403 and REST tenant isolation exercised.
Complexity and hardware-described benchmark	Complexity discussion included below; full reproducible benchmark not recorded.
Failure injection for duplicate/out-of-order/partial failure	Partial transaction failure and DB-unavailable recovery exercised; full suite incomplete.
Layered OOP design	Java Spring controllers, services, repositories, entities and client library used.
Trade-offs and omitted work	Documented in this report.
Monitoring metrics: lag, errors, retries, queue depth	Full operational metrics/monitoring suite not verified.
Caching and bounded staleness	Client snapshots and periodic refresh implemented; formal bound/eviction policy proof incomplete.
Error handling and non-leaking messages	Global exception handling implemented; full logging/security audit not recorded.
Design note and diagrams	Architecture, flow and failure table included; diagrams are described textually.
One-command run including database	Maven Wrapper instructions available; MySQL setup remains separate; Docker Compose not supplied.
README, public GitHub and demo video	README prepared; verify latest push. Demo video must be recorded and linked separately.

11. Complexity and scaling considerations

Let K be the number of registered keys, O the number of applicable overrides considered for a lookup, and C the number of connected SSE clients. A pre-resolved client map provides expected O(1) average lookup time and O(K) snapshot storage. Hierarchical server resolution checks at most four scope levels; with scope indexes, the hierarchy traversal is bounded by those four levels. Change fan-out is at least O(C) when one update is sent to C connected clients. Actual throughput and scaling limits depend on connection handling, database write rate, payload size and runtime resources. No capacity threshold is claimed without a reproducible benchmark.

12. Trade-offs and limitations

The design prioritizes low-latency local reads and continued availability by serving snapshots. The trade-off is temporary staleness while push or periodic refresh catches up. SSE is a straightforward server-to-client channel; periodic checks add recovery traffic. Persisted server snapshots and client snapshots improve resilience but do not prove recovery from every process-level failure. This report does not claim a hard five-second propagation guarantee, microsecond read benchmark, continuous-reader atomicity proof, complete duplicate/out-of-order proof, returned rollback diff, or production-grade monitoring.

13. Run instructions

Prerequisites: JDK 21 and MySQL 8. Create the database `tenant_config_db`. Set DB_PASSWORD in the same Windows CMD session, then run from the project root:

```
set DB_PASSWORD=your_local_mysql_password
mvnw.cmd clean compile
mvnw.cmd spring-boot:run
```

14. Final submission checklist

• Confirm the latest source code and README are pushed to the public GitHub repository.
• Ensure real secrets, local credentials and runtime snapshot files are not committed.
• Record a short demo showing hierarchy resolution, explain/history, atomic rejection, staged rollout/promotion, rollback, security/tenant isolation, live SSE update, and one failure scenario.

• Add the demo video link to the submission email.
• Do not describe the unverified requirements in this report as completed.

GitHub: https://github.com/ankitapaul74/tenant_config
Conclusion: The implementation demonstrates the main hierarchical configuration workflows, safe transactional rejection, staged rollout, rollback, SSE notification, snapshot recovery, and tenant-scoped REST access. The advanced-track proof requirements listed as unverified remain limitations of the current submission.